

Final project

Team 12 - Alon Bar, Amir Belinsky, Tal Ber and Nitzan Graf

“sda”

Figure 1: “sda”

Section 1 - Introduction

Context & Motivation

With the increasing importance of Large Language Models (LLMs), users and organizations face a critical optimization problem. Different tasks demand different skills, and some skills might be more expensive than others. For example: writing code versus answering advanced scientific questions. To optimize API costs, developers assign specific, simple tasks to cheaper models, while reserving complex reasoning for higher-end models. This market dynamic raises our core research question:

“Does \$1 buy you more coding skill, scientific reasoning, or general knowledge in today’s LLM market?”

Prior Work & Our Contribution

Existing platforms and leaderboards evaluate LLMs by comparing overall price against aggregate performance scores, treating intelligence as a single variable. This provides a general sense of model value, but ignores the nuances of specialized tasks. We build on this prior work by splitting “intelligence” into specific categories (coding, science, multidisciplinary reasoning). Rather than looking at overall scores, we analyze the isolated price per performance point in each specific category, while also comparing between open-source and closed-source models.

Why is this Hard?

Comparing cost and quality across different domains is analytically challenging. Because we compared slopes between different benchmarks, we faced many statistical problems. The first of them was making sure the different benchmarks are comparable. Then we had to check if the slopes we got were significant. Following that we had to make sure that the correlation we observed between cost and performance was actually meaningful, rather than a spurious correlation (meaning that there wasn’t another variable secretly affecting benchmark score and correlating to the costs).

Hypothesis

We hypothesize that the price-performance slope differs significantly across domains (for example, coding skills cost less than multi-disciplinary reasoning).

Section 2 - Data Overview

Data Sources and Entities

This analysis is based on two separate datasets. We joined Kaggle’s “LLM Price-Performance Tracker” (453 entities) with the “LifeArchitect.ai Models Table” (886 entities).

The LLM Price-Performance Tracker primary provides different benchmarks score and general model information like oss. The LifeArchitect dataset contains model architecture features like parameters number, token trained and architecture type.

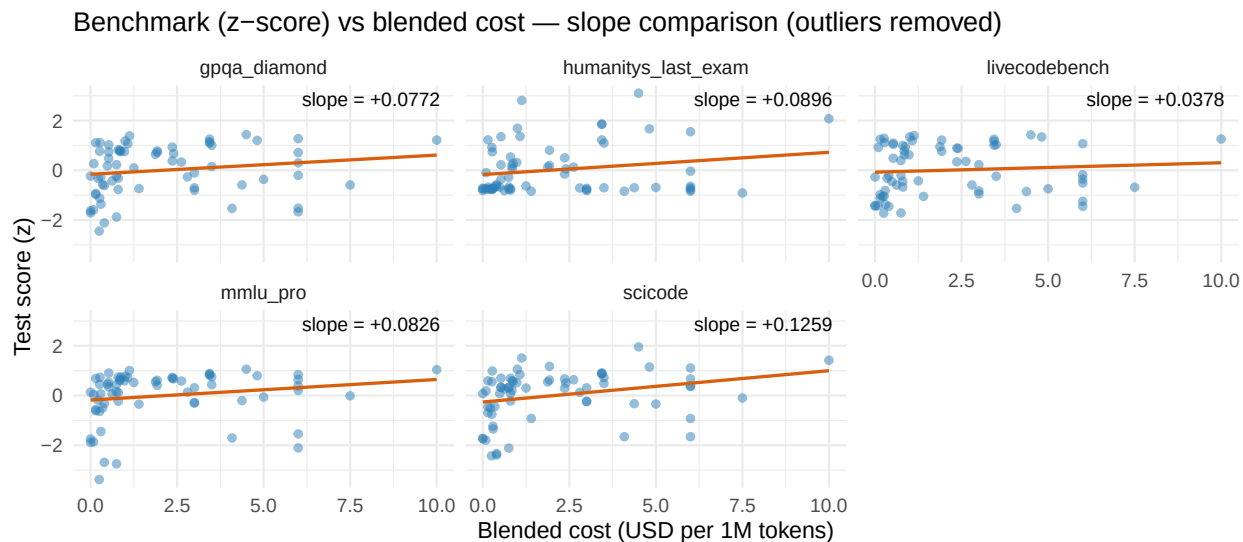
After deep data cleaning, we are left with 67 entities with 11 features.

Sample Selection

To get as many models as possible, with no null values in any row, we took the following steps: (1) choosing only the columns we need and removing the rest. (2) removing duplicate rows. (3) removing any row that had null values. (4) normalizing the “model_name” column so it matches with the LifeArchitect data-set.

Section 3 - Preliminary results

Comparison of benchmark score per blended_cost for each of the benchmarks.



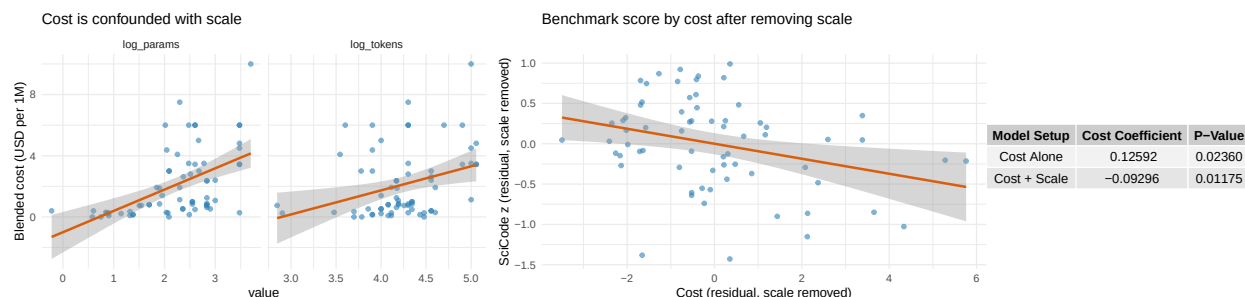
Here we perform a t-test on the significance of each slope. Only the slope for the scicode benchmark is statistically significant. That means that there isn't a clear correlation between the blended_costs to the benchmarks (apart from scicode).

Test	Slope	P-Value	R-Squared	Significant?
gpqa_diamond	0.0772338	0.1691984	0.0288763	FALSE
humanitys_last_exam	0.0895776	0.1100663	0.0388102	FALSE
livecodebench	0.0378465	0.5024630	0.0069481	FALSE
mmlu_pro	0.0826251	0.1421576	0.0328489	FALSE
scicode	0.1259248	0.0235975	0.0763619	TRUE

Here we figured out if cost is the real predictor. We showed the process only for scicode but its identical to the rest (we did the rest, this one is the most valuable). We have discovered a strong correlation between the the other parameters to the benchmark score, the parameters number for example (fig 1) and the table for each of the variables (fig 2). The we continued to find which variable alone predicts the benchmarks score the best (fig 3). We have found that there is a strong correlation between other variables and the benchmarks, and that the cost variable is very weak compared to the rest.



We see that the model’s “scale” (number of parameters and token trained) correlated very good with the cost (fig 1). After removing the scale of the model we have seen that the effect of the cost on the benchmark is actually negative (figs 2,3).

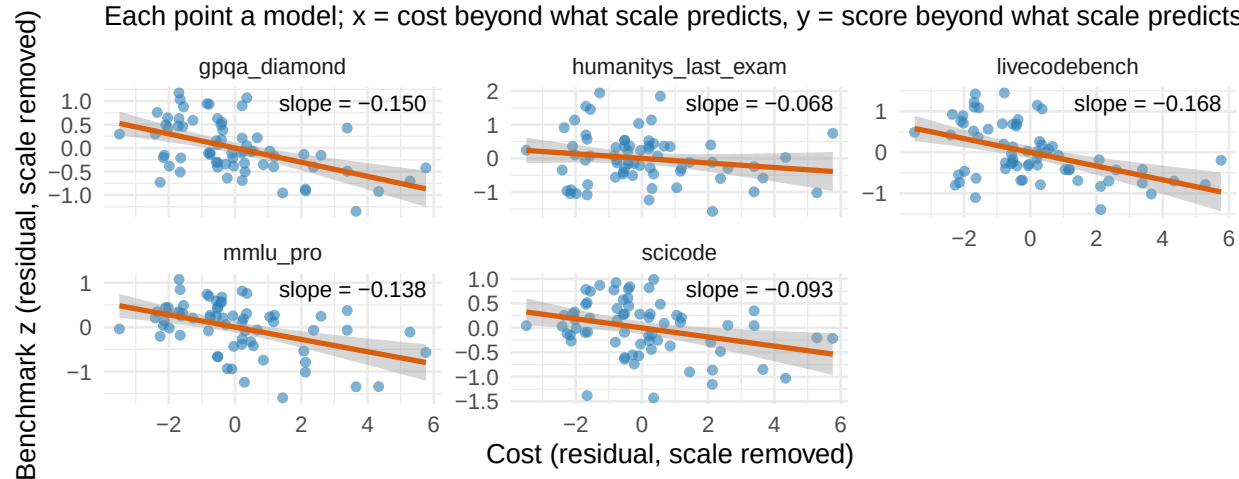


Final conclusion:

Once we hold the model scale constant, the cost variable stops measuring capability and starts measuring something closer to pricing strategy. Two models of the same scale can be priced differently, and our work shows that the cheaper models will usually perform better than the expensive ones. Across different domains, this translates to more aggressive pricing per scale, especially in livecodebench.

After removing scale, cost's effect on score is negative — all benchmarks

Each point a model; x = cost beyond what scale predicts, y = score beyond what scale predicts



Section 4 - Limitations and Future Work

Limitations - Sample Size: Merging the Kaggle and LifeArchitect datasets gave us valuable architectural variables, but reduced our final sample to 67 models. This smaller size slightly limits the statistical power of our regression analysis.

Time Sensitivity: Our data is a snapshot from March 2026. Because the AI market evolves rapidly, our specific price-per-performance estimates may become outdated.

Future Work - If given more time: We would analyze different pricing strategies preformed by the top companies and use it to enhance our research. We would try to find different pricing strategies across different domains and calculate how prices correlated to preformance inside each domain and pricing strategy.

Appendix

Data README

SISE2601 Project data description

=====

Team 12 - Alon Bar, Amir Belinsky, Tal Ber and Nitzan Graf

Main Data Description

Main Data File: llm_price_performance_tracker_2026-03-31.csv

Data Link: <https://www.kaggle.com/datasets/kanchana1990/llm-price-performance-tracker-march-2026>

Total Observations (Rows): 453

Total Variables (Columns): 34 / 8 (Variables kept)

model_name (Text): The official commercial or research name of the model.

mmlu_pro (Continuous Numerical): Massive Multitask Language Understanding Professional score.

gpqa_diamond (Continuous Numerical): Google-Proof Q&A benchmark score, representing advanced expertise.

humanitys_last_exam (Continuous Numerical): Performance on a highly complex benchmark designed to test

livecodebench (Continuous Numerical): Performance on real-world, dynamic programming and algorithmic challenges.

scicode (Continuous Numerical): Evaluation of the model's ability to solve scientific coding problems.

is_open_source (Binary): Indicates whether the model's weights are publicly accessible (TRUE) or restricted (FALSE).

blended_cost_usd_per_1m (Continuous Numerical): A weighted average cost per 1 million tokens, simulating

pricing_tier (Categorical): Free (0), Budget (<0.50), Mid (0.50-5), Premium (5-30), Ultra (>30), Unknown.

Second Data Description

Second Data File: LifeArchitect_Models.csv

Data Link: <https://lifearchitect.ai/models-table/>

Total Observations (Rows): 886

Total Variables (Columns): 47 / 3 (Variables kept)

model_name (Text): The official commercial or research name of the model.

Params (Continuous Numerical): The parameter count of the model.

Tokens trained (B) (Continuous Numerical): The amount of token used to train the model.

Arch (Categorical): The Architecture of the model (MoE, Dense)

Source code

```
# libraries
library(knitr)
library(tidyverse)
library(broom)
library(htmltools)
library(ggplot2)
library(dplyr)
library(tidyr)
library(car)
library(ppcor)
library(corrplot)
```

```

library(patchwork)
library(ggcorrplot)
library(gridExtra)
opts_chunk$set(echo=FALSE) # hide source code in the document

CSV_PATH_MAIN    <- "../data/llm_price_performance_tracker_2026-03-31.csv"
CSV_PATH_SECOND  <- "../data/LifeArchitect_Models.csv"

# columns to keep from each source
main_cols    <- c("model_name", "livecodebench", "scicode", "mmlu_pro",
                  "gpqa_diamond", "humanitys_last_exam", "is_open_source",
                  "blended_cost_usd_per_1m")
second_cols  <- c("model_name", "Params", "Tokens \ntrained (B)", "Arch")

# normalize model_name so the two sources match on the same key
norm <- function(x) sub(" (instruct|chat|base|it|preview|thinking)$", "",
                        gsub("\\s+", " ", trimws(gsub("[_]", " ",
                                                    tolower(sub("\\s*\\(.+\\.\\)", "", x))))))

df <- read_csv(CSV_PATH_MAIN, show_col_types = FALSE)
df_2 <- read_csv(CSV_PATH_SECOND, skip = 1, show_col_types = FALSE) # row 1 is junk
# Data cleaning

# Main dataset
df$model_name <- norm(df$model_name)
# keep only needed cols, then drop rows null in ANY of them
llm <- df %>% dplyr::select(all_of(main_cols)) %>% drop_na()
# one row per model: keep the strongest variant (highest livecodebench)
llm <- llm %>%
  group_by(model_name) %>%
  slice_max(livecodebench, n = 1, with_ties = FALSE) %>%
  ungroup()

# Second dataset
df_2$model_name <- norm(df_2$model_name)
# keep only needed cols, drop null rows, dedup
extra <- df_2 %>%
  dplyr::select(all_of(second_cols)) %>%
  drop_na() %>%
  distinct(model_name, .keep_all = TRUE)

# inner join
dataset <- inner_join(llm, extra, by = "model_name")

# normalize benchmarks
raw_kept <- c("livecodebench", "scicode", "mmlu_pro", "gpqa_diamond", "humanitys_last_exam")
zscore   <- function(x) (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
# add _z columns for each benchmark
for (c in raw_kept) {
  dataset[[paste0(c, "_z")]] <- zscore(dataset[[c]])
}

# cost cap for outliers filter (filters 2 very expensive models)

```

```

COST_CAP <- 20

# long format: one row per (model, test), outliers removed
plot_df <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  dplyr::select(blended_cost_usd_per_1m, ends_with("_z")) %>%
  pivot_longer(cols = ends_with("_z"),
               names_to = "test", values_to = "z") %>%
  mutate(test = sub("_z$", "", test))

# slope of z ~ cost per test (computed on the same filtered data)
slopes <- plot_df %>%
  group_by(test) %>%
  do(tidy(lm(z ~ blended_cost_usd_per_1m, data = .))) %>%
  filter(term == "blended_cost_usd_per_1m") %>%
  summarise(slope = estimate, .groups = "drop") %>%
  mutate(label = paste0("slope = ", sprintf("%.4f", slope)))

# faceted scatter + regression line, slope shown per panel
ggplot(plot_df, aes(x = blended_cost_usd_per_1m, y = z)) +
  geom_point(alpha = 0.5, size = 1.6, colour = "#2c7fb8") +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, colour = "#d95f0e", linewidth = 0.8) +
  geom_text(data = slopes, aes(x = Inf, y = Inf, label = label),
           hjust = 1.1, vjust = 1.5, size = 3.4, inherit.aes = FALSE) +
  facet_wrap(~ test, ncol = 3) +
  labs(title = "Benchmark (z-score) vs blended cost - slope comparison (outliers removed)",
       x = "Blended cost (USD per 1M tokens)",
       y = "Test score (z)") +
  theme_minimal(base_size = 12)

test_df <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  dplyr::select(blended_cost_usd_per_1m, ends_with("_z")) %>%
  pivot_longer(ends_with("_z"), names_to = "test", values_to = "z") %>%
  mutate(test = sub("_z$", "", test))

# fit z ~ cost per test; pull slope, p-value, R-squared
results <- test_df %>%
  group_by(test) %>%
  group_modify(~ {
    m <- lm(z ~ blended_cost_usd_per_1m, data = .x)
    tibble(
      slope = coef(m)[["blended_cost_usd_per_1m"]],
      p_value = glance(m)$p.value,
      r2 = glance(m)$r.squared
    )
  }) %>%
  ungroup() %>%
  mutate(significant_0.05 = p_value < 0.05)

# print(results)
# This renders a clean, compact table in your knitted document
kable(results,

```

```

    col.names = c("Test", "Slope", "P-Value", "R-Squared", "Significant?"),
    align = "lrrrrr") # Left align the name, right align the numbers

# prepare data frames
plot_df <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  mutate(log_params = log10(Params)) %>%
  dplyr::select(log_params, ends_with("_z")) %>%
  pivot_longer(cols = ends_with("_z"), names_to = "test", values_to = "z") %>%
  mutate(test = sub("_z$", "", test))

slopes <- plot_df %>%
  filter(test == "scicode") %>%
  do(tidy(lm(z ~ log_params, data = .))) %>%
  filter(term == "log_params") %>%
  summarise(slope = estimate, .groups = "drop") %>%
  mutate(label = paste0("slope = ", sprintf("%.4f", slope)))

md <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  mutate(
    log_params = log10(Params),
    log_tokens = log10(as.numeric(gsub(",", "", `Tokens \ntrained (B)`))),
    cost = blended_cost_usd_per_1m,
    oss = as.integer(is_open_source)
  ) %>%
  dplyr::select(scicode_z, log_params, log_tokens, cost, oss) %>%
  drop_na()

# correlation between scicode and parameters count
p1 <- ggplot(plot_df %>% filter(test == "scicode"), aes(x = log_params, y = z)) +
  geom_point(alpha = 0.5, size = 1.6, colour = "#2c7fb8") +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, colour = "#d95f0e", linewidth = 0.8) +
  geom_text(data = slopes, aes(x = Inf, y = Inf, label = label),
    hjust = 1.1, vjust = 1.5, size = 3.4, inherit.aes = FALSE) +
  labs(title = "SciCode Benchmark vs Model Size",
    x = expression(log[10]~"(Params, B)"),
    y = "Test score (z)") +
  theme_minimal(base_size = 11)

# heatmap of every variables
cor_matrix <- cor(md %>% dplyr::select(cost, log_params, log_tokens, oss))
p2 <- ggcorrplot(cor_matrix, lab = TRUE, lab_size = 3, tl.srt = 0,
  colors = c("#ef8a62", "white", "#67a9cf")) +
  theme(plot.margin = margin(10, 10, 10, 10), axis.text.x = element_text(angle = 45, hjust = 1))

# variable impact of prediction
single <- lapply(c("log_params", "log_tokens", "cost", "oss"), function(p) {
  m <- lm(reformulate(p, "scicode_z"), data = md)
  glance(m) %>% transmute(Predictor = p,
    `R-Squared` = round(r.squared, 3),
    `Adj. R-Squared` = round(adj.r.squared, 3),
    `P-Value` = format.pval(p.value, eps = 0.001))
})

```



```

}) %>% bind_rows()
p3 <- tableGrob(single, rows = NULL)

# put them in one row
(p1 + p2 + p3) + plot_layout(widths = c(1, 0.8, 1.3))
# new model
md <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  mutate(log_params = log10(Params),
         log_tokens = log10(as.numeric(gsub(",", "", `Tokens \ntrained (B)`))),
         cost = blended_cost_usd_per_1m,
         oss = as.integer(is_open_source)) %>%
  dplyr::select(scicode_z, log_params, log_tokens, cost, oss) %>%
  drop_na()

# model scale on costs
p1 <- md %>%
  pivot_longer(c(log_params, log_tokens),
              names_to = "predictor", values_to = "value") %>%
  ggplot(aes(x = value, y = cost)) +
  geom_point(alpha = 0.6, colour = "#2c7fb8") +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE, colour = "#d95f0e") +
  facet_wrap(~ predictor, scales = "free_x") +
  labs(title = "Cost is confounded with scale",
       y = "Blended cost (USD per 1M)") +
  theme_minimal(base_size = 12)

# raw vs scale-adjusted relationship of cost to score
md$score_resid <- resid(lm(scicode_z ~ log_params + log_tokens, data = md))
md$cost_resid <- resid(lm(cost ~ log_params + log_tokens, data = md))
p2 <- ggplot(md, aes(x = cost_resid, y = score_resid)) +
  geom_point(alpha = 0.6, colour = "#2c7fb8") +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE, colour = "#d95f0e") +
  labs(title = "Benchmark score by cost after removing scale",
       x = "Cost (residual, scale removed)",
       y = "SciCode z (residual, scale removed)") +
  theme_minimal(base_size = 12)

# cost's effect with vs without controlling for scale
zero_order <- lm(scicode_z ~ cost, data = md)
controlled <- lm(scicode_z ~ cost + log_params + log_tokens, data = md)

table_data <- bind_rows(
  tidy(zero_order) %>% filter(term == "cost") %>% mutate(model = "Cost Alone"),
  tidy(controlled) %>% filter(term == "cost") %>% mutate(model = "Cost + Scale")
) %>%
  dplyr::select(model, estimate, p.value) %>%
  mutate(estimate = round(estimate, 5), p.value = round(p.value, 5))

colnames(table_data) <- c("Model Setup", "Cost Coefficient", "P-Value")
p3 <- tableGrob(table_data, rows = NULL)

(p1 + p2 + p3) + plot_layout(widths = c(1.3, 1.3, 0.7))

```

```

# new model
md <- dataset %>%
  filter(blended_cost_usd_per_1m <= COST_CAP) %>%
  mutate(log_params = log10(Params),
         log_tokens = log10(as.numeric(gsub(",", "", `Tokens \ntrained (B)`))),
         cost = blended_cost_usd_per_1m,
         oss = as.integer(is_open_source)) %>%
  dplyr::select(all_of(paste0(raw_kept, "_z")), log_params, log_tokens, cost, oss) %>%
  drop_na()

# slope for every benchmark
cost_resid <- resid(lm(cost ~ log_params + log_tokens, data = md))

resid_df <- lapply(raw_kept, function(bench) {
  resp <- paste0(bench, "_z")
  sr <- resid(lm(reformulate(c("log_params", "log_tokens"), resp), data = md))
  tibble(benchmark = bench, cost_resid = cost_resid, score_resid = sr)
}) %>% bind_rows()

# per-benchmark slope label for the residual relationship
resid_slopes <- resid_df %>%
  group_by(benchmark) %>%
  summarise(slope = coef(lm(score_resid ~ cost_resid))[2], .groups = "drop") %>%
  mutate(label = paste0("slope = ", sprintf("%.3f", slope)))

ggplot(resid_df, aes(x = cost_resid, y = score_resid)) +
  geom_point(alpha = 0.6, size = 1.4, colour = "#2c7fb8") +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE, colour = "#d95f0e") +
  geom_text(data = resid_slopes, aes(x = Inf, y = Inf, label = label),
           hjust = 1.1, vjust = 1.5, size = 3, inherit.aes = FALSE) +
  facet_wrap(~ benchmark, ncol = 3, scales = "free_y") +
  labs(title = "After removing scale, cost's effect on score is negative - all benchmarks",
       subtitle = "Each point a model; x = cost beyond what scale predicts, y = score beyond what scale",
       x = "Cost (residual, scale removed)",
       y = "Benchmark z (residual, scale removed)") +
  theme_minimal(base_size = 11)

cat(readLines('../data/README.md'), sep = '\n')

```